

Chapter 2: Intelligent Agents

Agents, Environments, Rationality, and Agent Architectures

Based on Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition

Lecture Notes

Introduction to Artificial Intelligence

August 19, 2026

Lecture Roadmap

- 1 Agents and Environments
- 2 Rationality
- 3 Task Environments and PEAS
- 4 Properties of Task Environments
- 5 The Structure of Agents
- 6 Learning Agents
- 7 Representing States: Atomic, Factored, Structured
- 8 Summary

What Is an Agent?

Definition

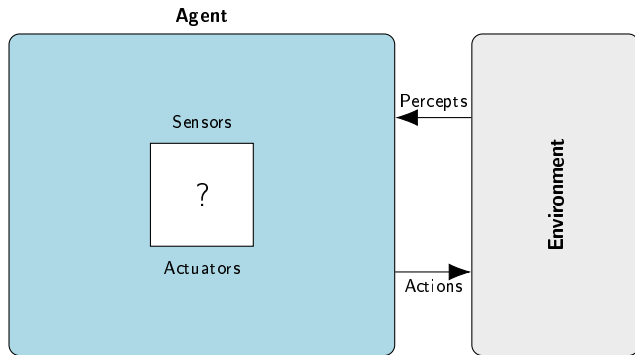
An **agent** is anything that can be viewed as *perceiving* its environment through **sensors** and *acting* upon that environment through **actuators**.

- **Human agent:** eyes, ears, skin, etc. are sensors; hands, legs, vocal tract, etc. are actuators.
- **Robotic agent:** cameras and infrared range finders as sensors; various motors as actuators.
- **Software agent (softbot):** receives file contents, network packets, keystrokes, mouse/voice input as percepts; acts by writing files, sending packets, displaying output, generating sound.

Key idea

The *environment* is, in principle, everything — but in practice it is only the part of the universe whose state we care about: the part that affects the agent's percepts and is affected by the agent's actions.

The Agent–Environment Loop



- Sensors deliver **percepts** to the agent from the environment.
- The agent processes percepts (the "?" box, to be filled in throughout the chapter) and chooses **actions**.
- Actuators carry out those actions, changing the environment.

Percepts and Percept Sequences

Percept

The content an agent's sensors are perceiving *at a given instant*.

Percept sequence

The *complete history* of everything the agent has ever perceived.

Crucial constraint

An agent's choice of action at any instant can depend on:

- its built-in (prior) knowledge, and
- the entire percept sequence observed so far,

but **not** on anything it has not perceived.

By specifying the action for *every possible percept sequence*, we specify everything there is to say about the agent.

The Agent Function

Definition

The **agent function** maps any given percept sequence to an action:

$$f : \mathcal{P}^* \longrightarrow \mathcal{A}$$

where $\mathcal{P}^*$ is the set of all possible percept sequences and $\mathcal{A}$ is the set of actions.

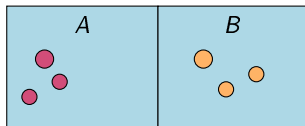
Two distinct ideas — do not confuse them:

- The **agent function** is an *abstract mathematical description* (external characterization) — in principle it could be tabulated by trying every possible percept sequence and recording the action taken.
- The **agent program** is a *concrete implementation* of that function, running on some physical architecture.

$$\text{agent} = \text{architecture} + \text{program}$$

Note: For most agents this table would be infinite unless we bound the percept-sequence length.

Case Study: The Vacuum-Cleaner World



- World consists of two locations: square *A* and square *B*.
- Each square can be **clean** or **dirty**.
- Agent perceives: *which square it is in* and *whether that square is dirty*.
- Agent starts in square *A*.
- **Available actions:** Right, Left, Suck, NoOp.

One simple agent function

If the current square is dirty, **Suck**; otherwise, move to the other square.

Partial Tabulation of the Vacuum Agent Function

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

- Different vacuum-world agents can be defined simply by *filling in the right-hand column differently*.
- The table is of **unbounded size** unless percept-sequence length is restricted.
- Central question: *What is the right way to fill out the table?* $\Rightarrow$ leads to **rationality**.

Agents Are a Modeling Tool, Not an Absolute Category

- The notion of “agent” is meant as a *tool for analysis*, not a strict line dividing the world into agents vs. non-agents.
- Example: A hand-held calculator could be viewed as an agent that outputs “4” given percept sequence “2 + 2 =” — but this analysis adds little understanding.
- All of engineering designs artifacts that interact with the world.
- AI focuses on the most interesting end of the spectrum: artifacts with
 - **significant computational resources**, and
 - **task environments requiring nontrivial decision making**.

Good Behavior: The Concept of Rationality

Rational agent

A **rational agent** is one that *does the right thing*.

But what does “the right thing” mean?

AI adopts consequentialism: we judge an agent’s behavior by its *consequences*.

- An agent generates a sequence of actions according to the percepts it receives.
- This causes the environment to pass through a sequence of **states**.
- If that sequence is desirable, the agent performed well.

Performance measure

An objective criterion for success; evaluates any given *sequence of environment states* (not the agent’s own opinion of itself).

Whose Preferences? Designer vs. Agent

- **Humans:** rationality concerns success in achieving states desirable *from the human's own point of view* — humans have innate desires/preferences.
- **Machines:** have no desires of their own. The performance measure is initially in the mind of the *designer* or the *user*.
- Some agent designs **explicitly represent** (a version of) the performance measure; in others it remains entirely **implicit** — the agent may do the right thing without “knowing why.”

Norbert Wiener's warning

Ensure that “the purpose put into the machine is the purpose which we really desire.” It can be surprisingly hard to formalize a performance measure correctly (the *King Midas problem*).

Pitfalls in Specifying Performance Measures

Naive proposal for the vacuum world: reward = amount of dirt cleaned in an 8-hour shift.

The problem

A rational agent can maximize *this* measure by:

- 1 cleaning up the dirt,
- 2 dumping it back on the floor,
- 3 cleaning it up again,
- 4 ...and repeating forever!

“What you ask for is what you get.”

A better measure

Reward the agent for having a *clean floor* — e.g., one point per clean square per time step (possibly with penalties for electricity used / noise generated).

General rule: Design performance measures according to what you actually want achieved *in the environment*, not according to how you think the agent should behave.

Deeper Subtleties in Performance Measures

Even avoiding the obvious traps leaves hard questions.

Example: “Clean floor” based on *average* cleanliness over time — but the same average can be achieved by:

- an agent that does a mediocre job constantly, or
- an agent that cleans energetically but takes long breaks.

Which is preferable? This is really a deep philosophical question:

- A reckless life of highs and lows vs. a safe, humdrum existence?
- An economy of moderate poverty for all vs. plenty for some and poverty for others?

Uncertainty about the true objective

Since designers cannot always anticipate every user's preferences, agents may need to represent **uncertainty about the performance measure itself** and learn more about it over time (see later chapters on value alignment).

What Rationality Depends On

Rationality at a given time depends on four things

- 1 The **performance measure** defining the criterion of success.
- 2 The agent's **prior knowledge** of the environment.
- 3 The **actions** the agent can perform.
- 4 The agent's **percept sequence to date**.

Definition of a Rational Agent

For each possible percept sequence, a rational agent should select an action that is expected to **maximize its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Worked Example: Is the Simple Vacuum Agent Rational?

Recall the agent function: *clean if dirty, otherwise move to the other square*. Is it rational?

Depends on the setup. Assume:

- Performance measure: 1 point per clean square per time step, over 1000 time steps.
- Geography (Fig. with squares *A, B*) known *a priori*; dirt distribution and initial location *unknown*.
- Clean squares stay clean; Suck cleans the current square.
- Right/Left move one square (or do nothing at a boundary).
- Only actions: Right, Left, Suck.
- Agent perceives location and local dirt correctly.

Result

Under these assumptions, the agent **is rational**: its expected performance is at least as good as any other agent's.

Same Agent, Different Assumptions — No Longer Rational

The *same* simple vacuum agent function can become irrational if the assumptions change:

But change the assumptions...

- If all dirt is cleaned and movement is penalized $\Rightarrow$ oscillating forever is **irrational**. A better agent does nothing once sure all squares are clean.
- If squares can become dirty again $\Rightarrow$ agent should periodically re-check.
- If geography is unknown $\Rightarrow$ agent must explore.

Takeaway

Rationality is always relative to the **performance measure**, **prior knowledge**, **available actions**, and **percept sequence** — change any one, and the same agent function may no longer be rational.

Rationality $\neq$ Omniscience

Omniscient agent

Knows the *actual* outcome of its actions and can act accordingly. **Impossible in reality.**

The Champs-Élysées example

You see an old friend across the street; no traffic nearby; you rationally start to cross. Meanwhile, at 33,000 feet, a cargo door falls off a passing airliner and flattens you before you reach the other side.

- Were you *irrational* to cross? **No** — your obituary would not read “Idiot attempts to cross street.”

Key distinction

Rationality maximizes *expected* performance (given available information).

Perfection maximizes *actual* performance (requires foreknowledge — crystal balls / time machines).

Rationality depends only on the percept sequence *to date*, not on unknowable future outcomes.

Information Gathering and Exploration

Does rationality excuse an agent from acting carelessly just because it *happened* not to perceive a danger?

No. Example: crossing a busy road without looking both ways.

- It would *not* be rational to cross given an uninformative percept sequence — the risk is too high.
- A rational agent should choose the **“looking” action** first, because looking helps maximize expected performance.

Information gathering

Doing actions in order to *modify future percepts* — an important part of rationality (Chapter 15 covers this formally).

Second example: A vacuum-cleaning agent in an initially unknown environment must **explore** to learn the geography before it can act optimally.

Rationality requires learning

A rational agent must not only gather information but **learn as much as possible** from what it perceives.

- Initial configuration may reflect *prior knowledge*, but experience should modify and augment it.
- **Extreme case:** If the environment is completely known *a priori* and fully predictable, the agent need not perceive or learn — it just acts correctly. Such agents are **fragile**.

The dung beetle

Plugs its nest with a ball of dung. If the dung ball is removed en route, the beetle continues the motions of plugging the nest with a *nonexistent* ball — it never notices. Evolution hard-wired an assumption; violating it causes failure.

The sphex wasp

Checks its burrow, drags in a caterpillar, and lays eggs. If an entomologist moves the caterpillar during the check, the wasp **restarts the plan from the check step**, repeatedly, even after dozens of interventions — it cannot learn its plan is failing.

Autonomy

Definition

To the extent an agent relies on the *prior knowledge of its designer* rather than on its own percepts and learning, it **lacks autonomy**.

- A rational agent **should be autonomous** — it should learn what it can to compensate for partial or incorrect prior knowledge.
- Example: A vacuum agent that *learns to predict* where/when dirt will appear outperforms one that does not.

Practical compromise

Complete autonomy from the very start is rarely required: with little/no experience, the agent would act nearly randomly unless the designer provides some initial assistance — just as evolution provides animals with built-in reflexes to survive long enough to learn.

After sufficient experience, a rational agent's behavior can become **effectively independent** of its prior knowledge. ⇒ Learning lets us design *one* agent that succeeds across a *vast variety* of environments.

Specifying the Task Environment: PEAS

Before building a rational agent, we must specify its **task environment** — the “problem” to which the rational agent is the “solution.”

PEAS

- **P**erformance measure
- **E**nvironment
- **A**ctuators
- **S**ensors

Golden rule

In designing an agent, the **first step** must always be to specify the task environment as fully as possible.

PEAS Example: The Automated Taxi Driver

Performance	Environment	Actuators	Sensors
Safe, fast, legal, comfortable trip; maximize profits; minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

- Many performance goals **conflict** (speed vs. safety vs. fuel economy) $\Rightarrow$ tradeoffs required.
- Environment ranges from rural lanes to 12-lane freeways; contains traffic, pedestrians, animals, roadworks, police, puddles, potholes.
- Optional design choices: geography (snow vs. no snow), driving side (right vs. left). **More restricted environment $\Rightarrow$ easier design problem.**

More PEAS Examples

Agent Type	Performance	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of Q's, tests, diagnoses, treatments	Touchscreen/voice entry of symptoms
Satellite image analysis	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-res digital camera
Part-picking robot	% parts in correct bins	Conveyor belt with parts, bins	Jointed arm and hand	Camera, tactile, joint-angle sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

Note: Virtual (software) task environments can be just as complex as the physical world — e.g., a **softbot** trading on auction websites interacts with millions of users and billions of objects.

Dimensions for Categorizing Task Environments

A fairly small number of dimensions largely determine the appropriate agent design:

- 1 Fully observable vs. partially observable (vs. unobservable)
- 2 Single-agent vs. multiagent
- 3 Deterministic vs. nondeterministic (vs. stochastic)
- 4 Episodic vs. sequential
- 5 Static vs. dynamic (vs. semidynamic)
- 6 Discrete vs. continuous
- 7 Known vs. unknown

These definitions are informal here; later chapters make them precise.

1. Fully Observable vs. Partially Observable

Fully observable

Sensors give access to the **complete state** of the environment at each point in time (or at least all aspects *relevant* to the choice of action).

- **Advantage:** the agent need not maintain any internal state to track the world.

Partially observable

Occurs due to noisy/inaccurate sensors, or because parts of the state are simply missing from sensor data.

- Example: a vacuum agent with only a *local* dirt sensor cannot tell whether the other square is dirty.
- Example: an automated taxi cannot see what other drivers are *thinking*.

Unobservable

If the agent has **no sensors at all**. Not necessarily hopeless — goals may still be achievable, sometimes even with certainty (Chapter 4).

2. Single-Agent vs. Multiagent

- Solving a crossword puzzle alone $\Rightarrow$ **single-agent**.
- Playing chess $\Rightarrow$ **two-agent (multiagent)**.

Subtlety: when must another entity be treated as an agent?

Key question: is entity B 's behavior best described as **maximizing a performance measure whose value depends on agent A 's behavior**? If yes, treat B as an agent; otherwise B can be treated as a stochastic/physical object.

- **Chess**: opponent B maximizes its own measure, which (by the rules) *minimizes* A 's measure $\Rightarrow$ **competitive** multiagent environment.
- **Taxi driving**: avoiding collisions maximizes *everyone's* performance $\Rightarrow$ partly **cooperative**; but only one car can occupy a parking space $\Rightarrow$ partly **competitive**.

Consequences for design:

- **Communication** often emerges as rational behavior in multiagent settings.
- In some competitive settings, **randomized behavior** is rational — it avoids predictability.

3. Deterministic vs. Nondeterministic (and Stochastic)

Deterministic

The next state of the environment is **completely determined** by the current state and the action(s) executed. Otherwise: **nondeterministic**.

- In a fully observable, deterministic environment, the agent need not worry about uncertainty.
- A **partially observable** environment can *appear* nondeterministic even if it is really deterministic.
- Most real situations are so complex that unobserved aspects must be treated as nondeterministic for practical purposes.

Taxi driving

Nondeterministic — other drivers' behavior can't be predicted exactly; tires blow out, engines seize.

Stochastic vs. nondeterministic — a technical distinction

- **Stochastic**: the model *explicitly* uses probabilities (“25% chance of rain”).
- **Nondeterministic**: possibilities are listed *without* being quantified (“a chance of rain”).

4. Episodic vs. Sequential

Episodic

Experience is divided into **atomic episodes**. Each episode: agent receives a percept, then performs a **single** action. The **next episode does not depend on prior actions**.

- Example: an agent that spots defective parts on an assembly line — each decision depends only on the current part.

Sequential

The current decision could affect **all future decisions**.

- Chess and taxi driving are sequential: short-term actions can have long-term consequences.

Why it matters

Episodic environments are **much simpler** — the agent does not need to think ahead.

5. Static vs. Dynamic (vs. Semidynamic)

Static

The environment **cannot change** while the agent is deliberating.

- Easy to handle: no need to keep watching the world while deciding, and no need to worry about the passage of time.

Dynamic

The environment **can change** while deliberating; it continuously asks “what do you want to do?” — not deciding counts as deciding to do nothing.

Semidynamic

The environment itself does not change with time, but the agent's **performance score** does.

Examples: Taxi driving is **dynamic**; Chess with a clock is **semidynamic**; Crossword puzzles are **static**.

6. Discrete vs. Continuous

This distinction applies to:

- the **state** of the environment,
- the way **time** is handled, and
- the **percepts** and **actions** of the agent.

Chess

Finite number of distinct states (ignoring the clock); discrete set of percepts and actions.

Taxi driving

Continuous-state, continuous-time: speed and location of the taxi and other vehicles sweep smoothly through continuous ranges. Steering angle is continuous too.

Note: Digital camera input is technically discrete but typically *treated* as representing continuously varying intensities/locations.

7. Known vs. Unknown

This is a property of the agent's/designer's *knowledge*, not of the environment itself

In a **known** environment, the outcomes (or outcome probabilities, if nondeterministic) for **all** actions are given. In an **unknown** environment, the agent must **learn** how it works to make good decisions.

Known/unknown is independent of fully/partially observable:

- **Known but partially observable:** solitaire — I know the rules, but can't see face-down cards.
- **Unknown but fully observable:** a new video game shows the entire game state on screen, but I don't know what the buttons do until I try them.

Performance measure can also be unknown

E.g., a taxi driver doesn't know if a new passenger wants a leisurely or speedy ride. This effectively makes the task environment **multiagent**, since the agent may need to learn preferences from interaction.

The Hardest Case

Hardest general case

Partially observable + multiagent + nondeterministic + sequential + dynamic + continuous + unknown.

- Taxi driving is hard in *almost* all these senses — except the driver's environment is **mostly known**.
- Driving a rented car in a new country (unfamiliar geography, different traffic laws, nervous passengers) is even more exciting!

Task Environments and Their Properties

Task Env.	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess w/ clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

- Properties are not always cut-and-dried (e.g., medical diagnosis could be considered multiagent if we model uncooperative patients/staff).
- AIMA's code repository provides environment simulators; we evaluate agents' **average performance** over an **environment class**, not a single environment.

Agent architecture

$$\text{agent} = \text{architecture} + \text{program}$$

- The **program** implements the agent function (mapping percepts to actions).
- The **architecture** is the physical computing device with sensors/actuators on which the program runs.
- The program must be appropriate for the architecture: if the program recommends `Walk`, the architecture had better have legs!
- Architecture ranges from an ordinary PC to a robotic car with multiple onboard computers, cameras, and sensors.

The Table-Driven Agent (and Why It Fails)

Simplest conceivable agent program: keep the entire percept history, look up the action in a giant table.

Why table-lookup is doomed

Let P = set of possible percepts, T = agent's lifetime (number of percepts received). Lookup table size:

$$\sum_{t=1}^T |P|^t$$

Numbers that make the point

- Automated taxi: one camera alone delivers ≈ 70 MB/s (30 fps, 1080×720 , 24-bit color) $\Rightarrow$ a lookup table with over $10^{600,000,000,000}$ entries for *one hour* of driving.
- Even chess — a tiny, well-behaved fragment of reality — has at least 10^{150} entries.
- The observable universe has fewer than 10^{80} atoms!

Why Table-Lookup Fails — The Verdict

- (a) No physical agent has room to *store* the table.
- (b) The designer would not have *time* to create it.
- (c) No agent could *learn* all the right entries from experience.

Yet, if correctly filled in, TABLE-DRIVEN-AGENT *does* implement the desired agent function — it's a useful conceptual baseline, just not a practical design.

Four Basic Kinds of Agent Programs

- 1 Simple reflex agents
- 2 Model-based reflex agents
- 3 Goal-based agents
- 4 Utility-based agents

Each can, in turn, be augmented into a **learning agent** that improves its own performance over time.

Definition

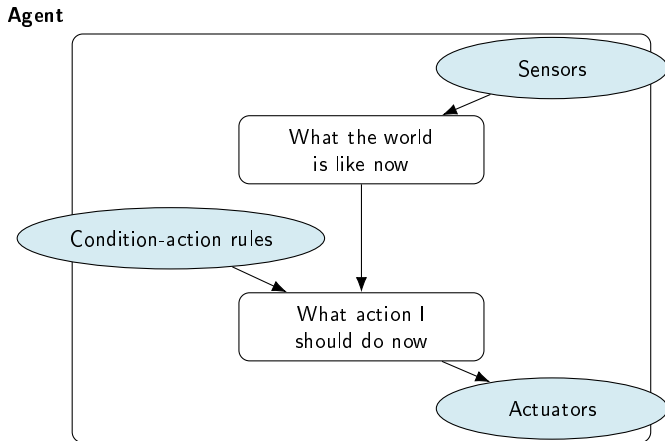
Select actions based **only on the current percept**, ignoring the rest of the percept history.

- Example: the vacuum agent of Figure 2.3/2.8 — decision depends only on current location & whether it is dirty.
- Implemented via **condition–action rules** (a.k.a. situation–action rules, productions, if–then rules):

if car-in-front-is-braking then initiate-braking

- Can be as simple as a Boolean circuit — humans have many such connections too (some learned, e.g. driving reflexes; some innate, e.g. blinking).

Simple Reflex Agent — Schematic



- **Rectangles** = current internal state of the decision process.
- **Ovals** = background information used in the process.

Pseudocode: SIMPLE-REFLEX-AGENT

REFLEX-VACUUM-AGENT([location, status])

```
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

SIMPLE-REFLEX-AGENT(percept)

```
persistent: rules -- a set of condition-action rules

state  <- INTERPRET-INPUT(percept)
rule   <- RULE-MATCH(state, rules)
action <- rule.ACTION
return action
```

- INTERPRET-INPUT generates an abstracted description of the current state.
- RULE-MATCH returns the first matching rule.
- “Rules” and “matching” are conceptual — could be a Boolean circuit or a neural net.

Limitations of Simple Reflex Agents

Works only if the environment is fully observable

The correct decision must be determinable from just the **current percept**.

Braking example

Rule: “if car-in-front-is-braking then initiate-braking” assumes brake lights are visible/identifiable from a single frame. Older cars with ambiguous taillight configurations break this assumption $\Rightarrow$ the agent brakes unnecessarily, or worse, never brakes.

Vacuum world without a location sensor

Only percepts: [*Dirty*] or [*Clean*]. On [*Clean*]: moving *Left* fails forever if starting in *A*; moving *Right* fails forever if starting in *B*.

Escaping Infinite Loops: Randomization

Randomization as an escape

On [*Clean*], flip a coin between `Right/Left`: reaches the other square in an average of 2 steps. A **randomized** simple reflex agent can outperform a deterministic one *in partially observable single-agent settings* — though in general, more sophisticated deterministic agents do much better.

Multiagent settings

Randomization is more clearly rational in *multiagent* environments (discussed earlier): it avoids the pitfalls of being predictable to an adversary. In **single-agent** environments, randomization is usually not rational — it is a useful trick in some situations, but sophisticated deterministic agents typically do much better.

Model-Based Reflex Agents

Key idea

Handle partial observability by **maintaining internal state** that tracks the part of the world the agent can't currently see, based on percept history.

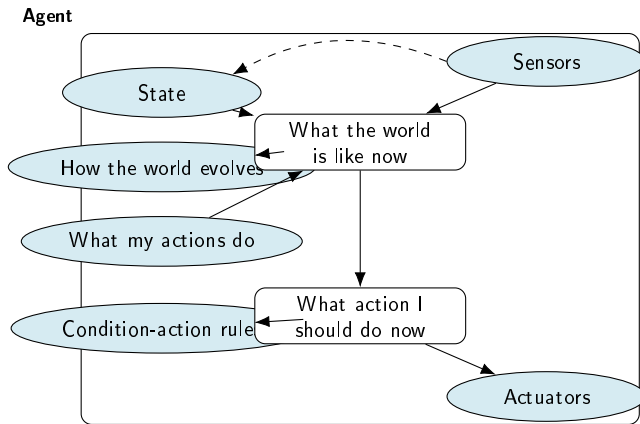
Example (braking): internal state can be as simple as the *previous camera frame*, letting the agent detect when both brake lights turn on/off simultaneously.

Two kinds of knowledge needed to update internal state:

- **Transition model:** how the world evolves — both the *effects of the agent's own actions* and how the world changes *independently*. (E.g., turning the wheel clockwise turns the car right; rain makes cameras wet.)
- **Sensor model:** how the state of the world is reflected in the agent's percepts. (E.g., braking in front $\Rightarrow$ red regions appear in the camera image; wet camera $\Rightarrow$ droplet artifacts obscure the road.)

An agent using such models is a **model-based agent**.

Model-Based Reflex Agent — Schematic



- Current percept combined with **old internal state** & the transition/sensor models → updated state description.
- UPDATE-STATE performs this fusion.

Pseudocode: MODEL-BASED-REFLEX-AGENT

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state      -- agent's current conception of world state
                    transition_model -- how next state depends on
                                current state and action
                    sensor_model   -- how current world state is
                                reflected in percepts
  rules                -- set of condition-action rules
  action               -- most recent action, initially none

  state <- UPDATE-STATE(state, action, percept,
                        transition_model, sensor_model)
  rule  <- RULE-MATCH(state, rules)
  action <- rule.ACTION
  return action
```

- It is **seldom possible** to determine the exact current state of a partially observable environment. “What the world is like now” represents the agent’s **best guess** (sometimes multiple guesses).
- Example: a taxi may not see around a stopped truck, and must *guess* the cause of the holdup.

Goal-Based Agents

Knowing the current state is not always enough

At a road junction: turn left, right, or go straight? The correct choice depends on **where the taxi is trying to get to**.

Goal

A description of a *desirable situation* — e.g., being at a particular destination.

- Combines the **model** (as in model-based agents) with **goal** information to choose actions that achieve the goal.
- Sometimes trivial (goal follows from a single action); sometimes requires considering **long action sequences** — this is the domain of **search** (Ch. 3, 4, 6) and **planning** (Ch. 11).

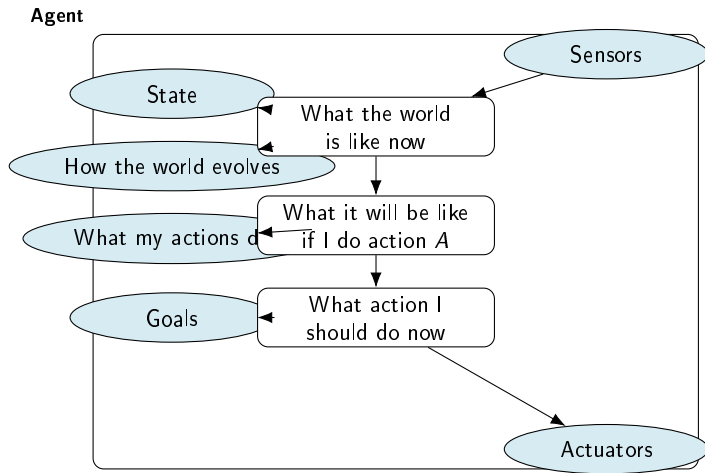
Goal-Based Agents: Why They're Different

Fundamentally different from reflex agents

Goal-based reasoning involves considering the *future*: “What will happen if I do such-and-such?” and “Will that make me happy?” Reflex agents brake when they see brake lights, **with no idea why**; a goal-based agent brakes because that action is predicted to achieve the goal of not hitting other cars.

Flexibility advantage: to change destination, a goal-based agent's goal is simply respecified; a reflex agent's *entire rule set* must be replaced.

Goal-Based Agent — Schematic



Note the **new component**: "What it will be like if I do action A" — explicit *lookahead*.

Utility-Based Agents

Goals alone are often not enough

Many action sequences achieve the goal (reach the destination), but some are quicker, safer, more reliable, or cheaper. Goals give only a **crude binary** distinction between “happy”/“unhappy” states.

Utility

A more general performance measure allowing comparison of *degrees* of happiness across world states. (Economists/CS use “utility” since “happy” isn’t scientific.)

Utility function

An agent’s **internalization** of the performance measure. If the internal utility function agrees with the external performance measure, maximizing utility $\Rightarrow$ rational behavior.

Not the only route to rationality — recall the rational vacuum agent (Fig. 2.8) that has *no* explicit utility function at all.

Why We Need Utility (Not Just Goals)

Two situations where **goals are inadequate** but utility-based agents still succeed:

- 1 **Conflicting goals** — only some achievable (e.g., speed *and* safety): the utility function specifies the appropriate **tradeoff**.
- 2 **Uncertain achievement** — several goals, none certain: utility **weighs likelihood of success against importance** of the goals.

Partial observability + nondeterminism are ubiquitous

⇒ decision making under uncertainty is unavoidable in the real world.

Expected utility

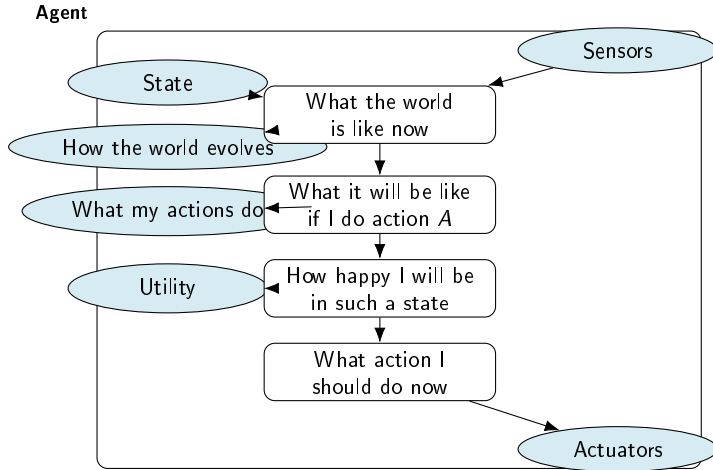
A rational utility-based agent chooses the action that **maximizes expected utility**:

$$EU(a) = \sum_{s'} P(s' \mid a) U(s')$$

i.e., utility averaged over possible outcomes, weighted by outcome probability.

Chapter 15 shows: any rational agent must behave *as if* it possesses a utility function whose expected value it tries to maximize — turning the “global” definition of rationality into a “local” constraint expressible in a simple program.

Utility-Based Agent — Schematic



Utility-Based Agents: Caveats

“Just maximize expected utility, and we’re done?” — Not so simple!

- Modeling and tracking the environment requires substantial research on **perception, representation, reasoning, learning** (fills much of the rest of the book).
- Choosing the utility-maximizing action is itself a hard computational problem (many chapters of algorithms).
- Even with good algorithms, **perfect rationality is usually unachievable in practice** due to computational complexity.

Model-free agents

Not all utility-based agents are model-based. A **model-free agent** can learn what action is best in a situation **without ever learning exactly how that action changes the environment** (Chapters 23, 26).

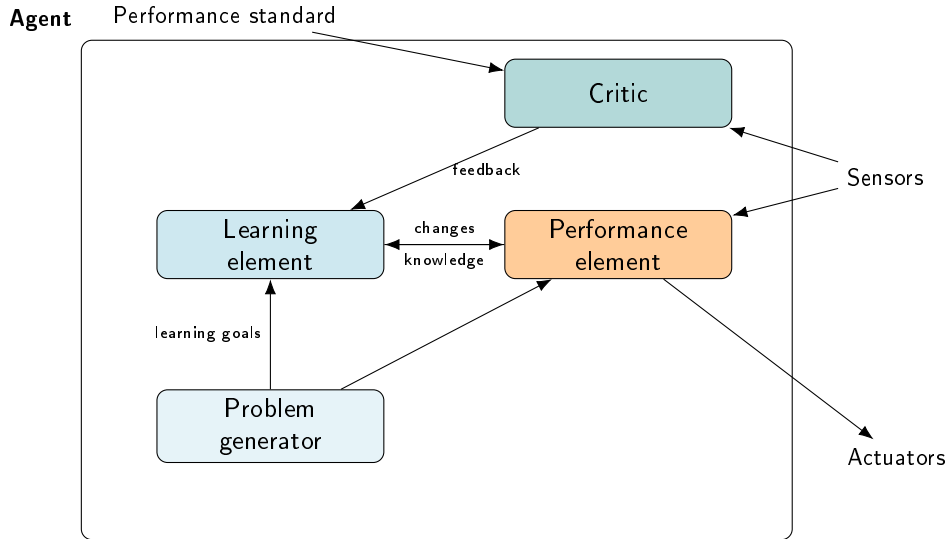
Caveat: All of this presumes the designer can specify the utility function correctly — Chapters 16, 17, 23 examine **unknown utility functions** in depth.

Turing's insight (1950)

Hand-programming intelligent machines directly is laborious. Turing concluded: “Some more expeditious method seems desirable” — namely, **build learning machines and then teach them**. This remains the preferred method for state-of-the-art AI systems today.

- **Any** type of agent (model-based, goal-based, utility-based) can be built as a learning agent (or not).
- Learning lets an agent operate in **initially unknown environments** and become **more competent** than its initial knowledge alone would allow.

The Four Components of a Learning Agent



Learning Agent Components, Explained (1/2)

Performance element

What we previously considered the **entire** agent: takes percepts, decides actions.

Learning element

Responsible for **making improvements**, using feedback from the **critic** to determine how the performance element should be modified.

Learning Agent Components, Explained (2/2)

Critic

Tells the learning element how well the agent is doing, w.r.t. a **fixed** external performance standard. *Necessary* because raw percepts don't indicate success (e.g., a chess program isn't told "checkmate = good" by the percept itself).

Problem generator

Suggests **exploratory actions** leading to new, informative experiences — even if suboptimal in the short run. Analogous to a scientist's experiments (e.g., Galileo dropping rocks from the Tower of Pisa to refine his theory of motion, not to break rocks!).

What Gets Learned, and From What Feedback

- The learning element can modify **any** “knowledge” component in the agent diagrams (state estimation, transition model, sensor model, condition-action rules, goals, utility function).
- **Simplest case:** learn directly from the percept sequence — observing pairs of successive states reveals “what my actions do” and “how the world evolves.”
 - E.g., taxi learns actual deceleration achieved on wet roads, and whether it skids.
- Improving the **model components** to better match reality is almost always beneficial, *regardless* of the external performance standard.
- Learning a **reflex component or utility function** *requires* information from the external performance standard.

Reward, Penalty, and Learning from Feedback

Reward and penalty

The performance standard singles out part of the incoming percept as a **reward** (or penalty) providing direct feedback on behavior quality — e.g., pain and hunger in animals.

Learning from human reactions

Taxi doesn't know people dislike loud noise; honks continuously. Passengers cover ears, complain, cut the horn wires — this human behavior gives the agent evidence to **update its utility function**.

Unifying theme

Learning = modifying each agent component to bring it into closer agreement with available feedback, improving overall performance.

How Do Agent Components Represent the World?

Representations lie along an axis of **increasing complexity and expressive power**:

Atomic $\longrightarrow$ **Factored** $\longrightarrow$ **Structured**

We illustrate with the component that answers “What do my actions do?”

Atomic Representation

Definition

Each state of the world is **indivisible** — a black box with no internal structure; the only discernible property is being identical to or different from another black box.

Example

Finding a driving route between cities: it may suffice to represent the state as just the *name of the city* we are in.

Used by: standard search and game-playing algorithms (Ch. 3, 4, 6), hidden Markov models (Ch. 14), Markov decision processes (Ch. 16).

Factored Representation

Definition

Each state is split into a **fixed set of variables/attributes**, each with a value (Boolean, real-valued, or one of a fixed set of symbols).

Example

Higher-fidelity driving-route description: gas level, GPS coordinates, oil-warning-light status, toll money available, radio station, etc.

- Two different *atomic* states share nothing in common — just different black boxes.
- Two different *factored* states can share **some** attributes (e.g., same GPS location) while differing in others (e.g., gas level) — makes it far easier to reason about transforming one state into another.

Used by: constraint satisfaction (Ch. 5), propositional logic (Ch. 7), planning (Ch. 11), Bayesian networks (Ch. 12–15, 18), many ML algorithms.

Structured Representation

Definition

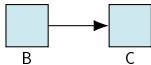
The world is described in terms of **objects**, each with attributes of its own **and** relationships to other objects.

Example

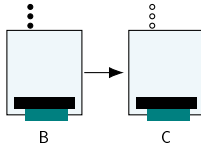
“A large truck ahead is reversing into a dairy-farm driveway, but a loose cow is blocking the truck’s path.”
A factored representation is unlikely to have a pre-defined attribute like `TruckAheadBackingIntoDairyFarmDrivewayBlockedByLooseCow`. We need explicit objects (*truck*, *cow*, *driveway*) and relationships among them.

Used by: relational databases, first-order logic (Ch. 8–10), first-order probability models (Ch. 18), much of natural language understanding (Ch. 24, 25).

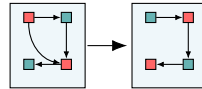
Three Representations, Side by Side



(a) Atomic



(b) Factored



(c) Structured

(a) State = opaque black box. (b) State = vector of attribute values. (c) State = objects with attributes *and* relationships.

Expressiveness Trade-offs

Expressiveness

A more expressive representation can capture **everything** a less expressive one can, *plus more* — usually much more **concisely**.

Chess rules, three ways

- First-order logic (structured): ~1–2 pages.
- Propositional logic (factored): thousands of pages.
- Finite-state automaton (atomic): around 10^{38} pages!

The catch

Reasoning and learning become **more complex** as expressive power increases. Real-world intelligent systems may need to operate at **all points along the axis simultaneously** to get the benefits of expressiveness while managing complexity.

Localist vs. Distributed Representations

A second, orthogonal axis: how concepts map onto **physical memory** (computer or brain).

Localist representation

A **one-to-one** mapping between concepts and memory locations.

- Fragile: if a transmission error garbles a few bits, **Truck** could be confused with the unrelated concept **Truce**.

Distributed representation

The representation of a concept is **spread over many** memory locations, and each location participates in representing **multiple** concepts.

- More **robust** against noise/information loss: think of each concept as a point in multidimensional space — garbling a few bits moves you to a *nearby* point with similar meaning.

Chapter Summary (1/2)

- An **agent** perceives and acts in an environment; the **agent function** specifies the action for any percept sequence.
- The **performance measure** evaluates a sequence of environment states. A **rational agent** acts to maximize *expected* performance measure, given the percept sequence seen so far.
- A **task environment** specification = **PEAS**: Performance, Environment, Actuators, Sensors. Always specify this *first* when designing an agent.
- Task environments vary along key dimensions: fully/partially observable, single/multiagent, deterministic/nondeterministic, episodic/sequential, static/dynamic, discrete/continuous, known/unknown.

Chapter Summary (2/2)

- If the performance measure is unknown or hard to specify, there's real risk of optimizing the **wrong objective** — the agent design should reflect **uncertainty** about the true objective.
- The **agent program** implements the agent function. Four canonical designs, in increasing sophistication:
 - **Simple reflex** agents (react to current percept only)
 - **Model-based reflex** agents (track hidden state via transition & sensor models)
 - **Goal-based** agents (choose actions to achieve explicit goals)
 - **Utility-based** agents (maximize expected utility)
- **Any** of these can be extended into a **learning agent** (performance element + learning element + critic + problem generator) to improve with experience.
- World states can be represented as **atomic**, **factored**, or **structured** — a trade-off between simplicity and expressive power.

Key Terms Checklist

Agent, sensor, actuator, environment

Agent program, agent architecture

Consequentialism

Fully/partially/unobservable

Deterministic/nondeterministic/stochastic

Static/dynamic/semidynamic

Known/unknown environment

Simple reflex agent, condition-action rule

Goal-based agent

Model-free agent

Reward / penalty

Expressiveness

Percept, percept sequence, agent function

Rational agent, performance measure

PEAS

Single-agent/multiagent, competitive/cooperative

Episodic/sequential

Discrete/continuous

Table-driven agent

Model-based reflex agent, transition model, sensor model

Utility-based agent, utility function, expected utility

Learning agent: performance element, critic, learning element, problem generator

Atomic / factored / structured representation

Localist / distributed representation

Discussion Questions for Class

- 1 Give a PEAS description for an agent that plays online chess anonymously against unknown opponents. Classify each environment property.
- 2 Construct an example of a performance measure that is easy to state but leads to unintended (“reward-hacking”) agent behavior, in the spirit of the vacuum-cleaner example.
- 3 Why is a rational agent not required to be omniscient? Give a real-world example (other than the textbook’s) where a rational choice led to a bad outcome.
- 4 For an email spam-filtering agent, sketch the four learning-agent components: performance element, critic, learning element, problem generator.
- 5 Compare atomic, factored, and structured representations for the *same* task environment of your choosing. What is gained and lost as you move from atomic to structured?

- **Chapters 3–4:** Search-based problem solving for goal-based agents (atomic representations).
- **Chapter 5:** Constraint satisfaction problems (factored representations).
- **Chapter 6:** Adversarial search and games (multiagent, competitive environments).
- **Chapter 11:** Planning (factored representations, goal-based agents).
- **Chapters 12–18:** Uncertainty, probabilistic reasoning, and utility-based / decision-theoretic agents.
- **Chapters 19, 21–23:** Learning algorithms in depth.

Next lecture: Solving Problems by Searching (Chapter 3)